

QUIZARD FEATURES STORYBOOK

Master Architecture, Gameplay Mechanics, and Tournament Systems

A Comprehensive Technical Specification and Narrative Guide for Decision Makers, Product Managers, and Backend Software Engineers

Core System Pillars Covered

- **Dynamic Rules Governance:** Server-driven timers (180s), 60-question sets, and 2-attempt daily limits
- **High-Speed Question Engine:** Sub-50ms ID sampling, difficulty weighting, and anti-memorization exclusions
- **Military-Grade Cryptography:** AES-256-CBC payload encryption and anti-cheat answering velocity verification
- **Live Leaderboards & Tie-Breaking:** Real-time SQL window ranking and strict time-based tie resolution
- **Automated Tournament Crons:** 04:30 AM daily billing verification and Thursday 05:30 AM weekly championships
- **Clean Architecture Migration:** Transitioning from Django REST to high-throughput .NET 10 on PostgreSQL
- **High-Velocity Telemetry Gateway:** Dedicated FastAPI microservice (cms.quizard.live) on MongoDB & Redis
- **CMS Operations & Telco Billing:** Question approval lifecycles, Wordly, Speed Math, DOB, and bKash passes
- **Financial Reporting & BI Engine:** Multi-tier winner audits, automated bKash B2C payouts, and revenue analytics
- **Score-Driven Performance Bands:** 6 score-driven tiers (0 to 60) with asymmetric delta adjustments
- **XP Ledger & Linear Streaks:** Linear daily streak progression ($+20 \text{ XP} \times n$) with giant Day 15 milestone surge
- **Rank Velocity & Adaptive Challenge:** Live $\uparrow / \downarrow$ glyphs, personalized cards, and 10s fast timers

Author: Engineering Architecture Team

Ecosystem: Next.js 16 Frontend $\longleftrightarrow$ Quizard-API (FastAPI / MongoDB) $\longleftrightarrow$.NET 10
(PostgreSQL)

Database Cluster: PostgreSQL Enterprise Cluster (Host: 103.174.50.155) + MongoDB
Telemetry Cluster

Date: August 2026

Contents

Executive Overview: The Quizard Platform	4
1 The Rules Engine: Game Governance, Timers, and Attempt Limits	5
1.1 Why Dynamic Rules Matter	5
1.2 The Player Journey: Rahim Checks Today's Tournament Rules	6
1.2.1 Morning Setup in Dhaka	6
1.2.2 Enforcing the Daily Attempt Cap	6
2 The Question Engine: Selection, Difficulty Weighting, and Real-Time Delivery	8
2.1 The Architecture of the Question Bank	8
2.2 The Question Selection Algorithm	8
2.2.1 Why Fast ID Sampling Beats Full Database Sorting	9
2.3 The Player Journey: Karim Plays a 60-Question Rapid Match	9
2.3.1 Session Launch in Chittagong	9
2.3.2 Racing Against the 180-Second Clock	10
3 The Shield of Integrity: Cryptography, Anti-Cheat, and Score Persistence	11
3.1 The End-to-End Cryptographic Handshake	11
3.2 The Multi-Layer Anti-Cheat Verification Suite	12
3.3 Two Divergent Journeys: Salma vs. Tanvir	12
3.3.1 Salma's Legitimate Victory (59/60 in 150 Seconds)	12
3.3.2 Tanvir's Automated Exploit Attempt (Flagged & Blocked)	13
4 The Hall of Champions: Live Leaderboards and Tournament Consolidation	14
4.1 The Real-Time Daily Leaderboard Engine	14
4.1.1 Strict Tie-Breaking Hierarchy	15
4.2 The Overnight Daily Verification Pipeline (04:30 AM Cron)	15
4.3 The Weekly Tournament Championship (Thursday 05:30 AM)	15
4.3.1 The 7-Day Rolling Tournament Cycle	15
4.3.2 Stored Procedure Architecture	16
4.4 The Player Journey: Ayesha Wins the Weekly Championship	16
5 Architecture Blueprint: Clean Architecture and Database Schemas	18
5.1 The .NET 10 Clean Architecture Structure	18
5.2 Comprehensive API Endpoint and Schema Mapping	18
5.3 Critical PostgreSQL Table Schemas and Indexing	18
6 The Gateway of Speed: The FastAPI Microservice and CMS Engine	20
6.1 The Need for a Dedicated High-Velocity Microservice	20
6.2 Core Responsibilities of Quizard-API	21
6.3 Real-World Stories and Concrete Examples	21

6.3.1	Story 1: Rahim's Campaign Click and Asynchronous Telemetry	21
6.3.2	Story 2: Marketing Executives Analyze Hourly Campaign ROI	22
6.3.3	Story 3: Karim Fetches Instant Cached Rules	22
7	The Master Ecosystem: CMS Operations, Multi-Game Engines, and Telco Billing	24
7.1	The CMS Question Authoring and Approval Pipeline	24
7.1.1	Data Model of a CMS Question Entity	24
7.2	Admin Governance and Role-Based Access Control (RBAC)	25
7.3	The Multi-Game Suite: Beyond Rapid Trivia	25
7.3.1	1. Wordly: The Daily Bengali & English Word Puzzle	25
7.3.2	2. Speed Math and Addition Sprint	25
7.3.3	3. Memory Match and Sudoku Challenges	25
7.4	Direct Operator Billing (DOB) and Telco Subscriptions	26
7.5	Financial Operations, Referrals, and Prize Disbursements	26
7.6	Ad Network Attribution and Campaign Tracking	26
7.7	Real-World Stories: Behind the Scenes	27
7.7.1	Story 1: Farhana Prepares the National BCS Tournament Questions . . .	27
7.7.2	Story 2: Rupa Wins the Daily Wordly Challenge in Sylhet	27
7.7.3	Story 3: Monir Audits and Disburses Weekly Prizes	27
8	The Financial and Analytics Hub: Reporting Engine, Payouts, and Business Intelligence	28
8.1	The Architecture of the Reporting Engine	28
8.2	Core Modules of the Reporting Suite	28
8.2.1	1. Daily and Weekly Tournament Winner Reports	28
8.2.2	2. Micro-Prize Tier Reporting (5 TK and 10 TK Bounties)	29
8.2.3	3. Wordly Word Puzzle Winner Analytics	29
8.2.4	4. Referral Tracking and Bounties (<code>PrizeOnReferral</code>)	29
8.2.5	5. Churn-Back and Win-Back Analytics (<code>ChurnBackReportViewSet</code>) . . .	29
8.2.6	6. Real-Time Revenue and Charging Graphs (<code>apps.allReportGraph</code>) . .	29
8.3	Real-World Scenario: Tanima Audits and Executes Weekly Payouts	29
9	The Performance Rating Architecture: 6 Score Bands, Dynamic Deltas, and Progression Modeling	31
9.1	The 6 Integer Performance Bands (0 to 60 Scale)	31
9.2	The Integer Dynamic Rating Update Formula	31
9.2.1	Core Principles of the Integer Engine	32
9.3	Step-by-Step Scenarios: The Player at Rating 34	32
9.3.1	Detailed Analysis of Each Case	32
9.4	Multi-Round Lifecycle Simulation (Pure Integers 0 to 60)	33
9.5	Database Schema and API Architecture	33
9.5.1	1. PostgreSQL Historical Rating Table	33
9.5.2	2. REST Endpoints Specification	34
10	The XP Ledger, Linear Daily Streaks, and Milestone Achievement System	35
10.1	Core Experience Point (XP) Earning Mechanics	35
10.2	Linear Daily Streaks with the Giant 15th-Day Surge	35
10.2.1	Linear Daily Formula	35
10.2.2	The 15-Day Milestone Progression Matrix	35
10.3	Milestone Achievement Badges Shelf	35
10.4	Database Schema and API Architecture	37
10.4.1	1. PostgreSQL Immutable XP Ledger Table	37
10.4.2	2. REST Endpoints Specification	37

10.5 Real-World Scenario: Sabina's 15-Day Streak Journey	37
11 Rank Velocity, Profile Personalization, and Adaptive Live Experiences	38
11.1 Real-Time Leaderboard Rank Velocity Indicators	38
11.2 Server-Driven Profile Personalization	38
11.3 Adaptive Difficulty and High-Experience Fast Timers	38
11.4 Database Schema and API Architecture	39
11.4.1 1. REST Endpoints Specification	39
11.5 Real-World Scenario: Rafiq's Leaderboard Surge	39
12 Technical Glossary and Reference Guide	41
12.1 Glossary of Core Domain Entities	41

Executive Overview: The Quizard Platform

Platform Mission

Quizard is Bangladesh's premier real-time interactive quiz and tournament platform, engaging hundreds of thousands of concurrent players across web and telecommunication partner channels (bKash, Banglalink, Robi).

To ensure absolute competitive integrity, fair prize distribution, and instantaneous responsiveness, Quizard combines server-authoritative game governance, sub-50ms dynamic question generation, end-to-end payload cryptography, and automated tournament consolidation.

The End-to-End System Architecture

The diagram below illustrates the comprehensive lifecycle of a player's journey through Quizard:

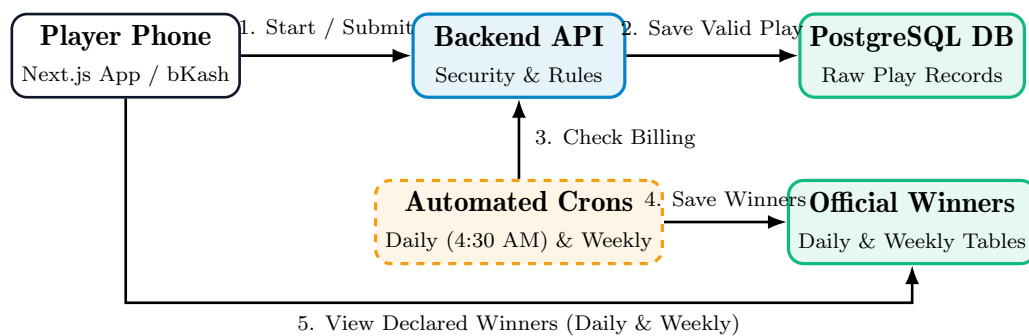


Figure 1: Quizard End-to-End Competition and Winner Lifecycle

Chapter 1

The Rules Engine: Game Governance, Timers, and Attempt Limits

The Rules Engine in Brief

Before a single question appears on a player's phone screen, the Quizard backend consults its **Rules Engine**. The Rules Engine acts as the tournament constitution: it dynamically sets how many questions each player receives (e.g., 60 questions), how many seconds they have to answer them (e.g., 180 seconds / 3 minutes), how many attempts they are allowed per day (e.g., exactly 2 rounds per day), and what reward prizes are at stake.

1.1 Why Dynamic Rules Matter

In traditional mobile games, quiz rules and timers are often hardcoded directly into the phone app. If tournament organizers want to change a quiz duration from 3 minutes to 2 minutes for a special holiday tournament, they would normally have to build and publish a brand new app update to the Google Play Store and Apple App Store.

Quizard solves this through its **Backend-Driven Rules Engine** (mapped in Django to `apps.cms.models.Rules` and in .NET to the `Rule` domain entity). Every aspect of gameplay is determined on the server in real time:

- **Allocated Time** (`allocated_time`): Total gameplay window in seconds (e.g., 180 seconds = 3 minutes).
- **Question Set Count** (`question_set`): Total questions served per quiz round (e.g., 60 questions).
- **Daily Round Limit** (`round`): Maximum allowed plays per calendar day (e.g., 2 attempts per day).
- **Passing Score** (`passing_score`): Minimum correct answers required to qualify for daily rewards (e.g., 20 correct answers).
- **Category Distribution** (`category`): Array of approved topic IDs (General Knowledge, BCS Preparation, Sports, Science, Islamic History).
- **Reward Prize Pool** (`winningPrize`): Daily, weekly, or special event cash prizes (e.g., 5,000 BDT to 25,000 BDT).

1.2 The Player Journey: Rahim Checks Today's Tournament Rules

1.2.1 Morning Setup in Dhaka

It is 08:30 AM in Dhaka. Rahim, an avid university student and competitive quiz enthusiast, opens the Quizard mobile web application on his smartphone.

Before Rahim sees the "Start Quiz" button, his phone sends a background request to the backend:

GET /api/rules/?portal=15&events=34



Figure 1.1: The Dynamic Rules Retrieval Lifecycle

The backend retrieves the active configuration for Event 34 on Portal 15 (bKash/Quizard Live) and delivers the JSON configuration:

Listing 1.1: Rules Engine JSON Payload for 60-Question Tournament

```

1 {
2   "id": 34,
3   "events": "Quizard Mega Rapid Tournament",
4   "category": [1, 2, 3, 4, 5],
5   "question_set": 60,
6   "allocated_time": 180,
7   "round": 2,
8   "passing_score": 20,
9   "winningPrize": "10,000 BDT",
10  "status": "active",
11  "company": 15
12 }
  
```

1.2.2 Enforcing the Daily Attempt Cap

Rahim knows he has ****2 attempts per day**** for this tournament.

- 1. First Attempt at 09:00 AM:** Rahim plays his first round. He answers 60 rapid questions, scoring **54 correct** in **145 seconds**.
- 2. Second Attempt at 02:00 PM:** Seeking to beat his score, Rahim plays his second round. He scores **58 correct** in **140 seconds**.
- 3. Third Attempt at 06:00 PM (Blocked):** When Rahim attempts to start a third round, the backend counts his completed records for the day:

$$\text{Daily Attempt Count} = 2 \geq \text{Allocated Rounds (2)}$$

The backend returns an informative HTTP 403 response: "Daily round limit reached. Please return tomorrow at 12:00 AM for fresh attempts."

Field	Type	Example	Engine Responsibility
allocated_time	Integer (Sec)	180	Sets countdown clock for 60 questions (3 minutes total).
question_set	Integer	60	Specifies exactly how many questions to pull from category pools.
round	Integer	2	Daily attempt limit per MSISDN per calendar day.
passing_score	Integer	20	Minimum score to qualify for rewards and tournament certificates.
category	List[Int]	[1, 2, 4]	Filters question selection to approved tournament topics.
winningPrize	Varchar	10,000 BDT	Declares the official monetary prize for top rankers.

Table 1.1: Key Fields in the Quizard Rules Governance Engine

Chapter Key Takeaways

- Quizard decouples gameplay rules from client applications, empowering administrators to configure tournament parameters instantly on the server.
- A standard rapid tournament serves **60 questions** within an **allocated time of 180 seconds** (3 minutes).
- The backend strictly validates daily attempt limits (e.g., 2 rounds per day), preventing brute-force spamming while maintaining competitive fairness across Bangladesh.

Chapter 2

The Question Engine: Selection, Difficulty Weighting, and Real-Time Delivery

The Question Engine in Brief

How does Quizard generate a unique, balanced set of 60 rapid questions in less than 50 milliseconds when thousands of players hit “Start Quiz” simultaneously? The Question Engine combines category filtering, anti-memorization exclusions, multi-tier difficulty weighting, and sub-millisecond database ID sampling to serve exciting and fair quiz sessions to every contestant.

2.1 The Architecture of the Question Bank

In the Quizard database (mapped to `apps.cms.models.Questions`), tens of thousands of approved questions are curated across diverse categories. Each question entity contains:

- **Question Text** (`question`): The Bengali or English trivia query.
- **Four Options** (`option1`, `option2`, `option3`, `option4`): The multiple-choice choices presented to the contestant.
- **Correct Answer** (`ans`): The authoritative correct answer string (kept confidential on the server or sanitized before client transmission).
- **Category Identifier** (`category_id`): Topic tag (e.g., General Knowledge, BCS, Sports, Science, Literature, Islamic History).
- **Difficulty Weightage** (`weightage`): Integer ranging from 1 (Foundational/Easy) to 6 (Expert/Hard).
- **Portal Identifier** (`company_id`): Tenant isolation ensuring custom partner content (e.g., Portal 15 for bKash/Quizard Live, Portal 20 for Banglalink, Portal 21 for Robi) remains segregated.

2.2 The Question Selection Algorithm

When a contestant initiates a quiz via `GET /api/questions/?rules=34&portal=15`, the Question Engine executes a multi-step pipeline:

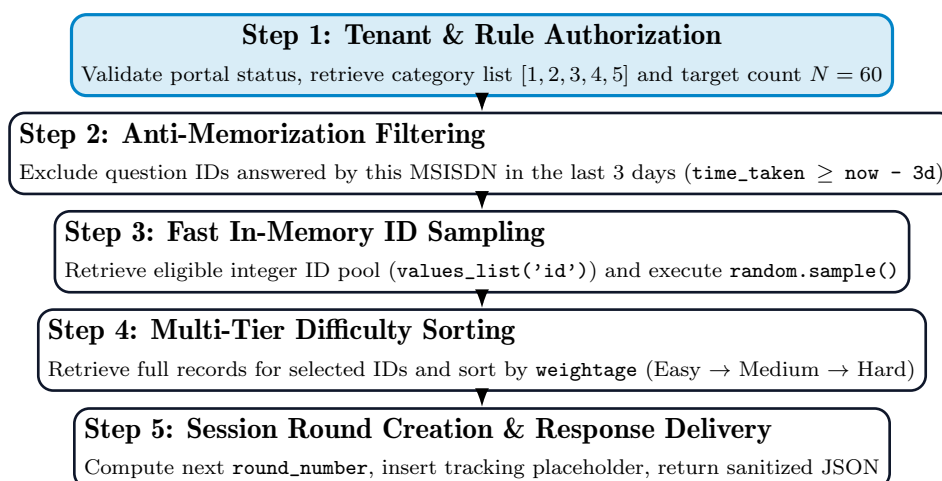


Figure 2.1: The 5-Stage Dynamic Question Selection Pipeline

2.2.1 Why Fast ID Sampling Beats Full Database Sorting

Naively executing `ORDER BY RAND() LIMIT 60` across a database table with 50,000 rows causes full-table scans, disk thrashing, and database CPU spikes during peak traffic.

Quizard achieves instant sub-50ms question serving by pulling only lightweight 4-byte integer IDs into memory:

Listing 2.1: Fast ID Sampling Algorithm in Django Backend

```

1 # Step 1: Fetch lightweight ID list for approved categories
2 all_ids = list(Questions.objects.filter(
3     company_id=portal_id,
4     category_id__in=approved_categories,
5     status='approved'
6 ).values_list('id', flat=True))
7
8 # Step 2: High-speed O(K) pseudo-random sampling in memory
9 selected_ids = random.sample(all_ids, min(question_count,
10     len(all_ids)))
11
12 # Step 3: Fetch only the 60 selected records by Primary Key
13 selected_questions =
14     list(Questions.objects.filter(id__in=selected_ids))
15
16 # Step 4: Sort progressively by weightage for smooth difficulty ramp
17 selected_questions.sort(key=lambda q: q.weightage)
  
```

2.3 The Player Journey: Karim Plays a 60-Question Rapid Match

2.3.1 Session Launch in Chittagong

In Chittagong, Karim taps “Play Tournament”. His phone sends his authentication bearer token to the backend:

```
GET /api/questions/?rules=34&portal=15
```

The backend detects that Karim has played 0 rounds today. It calculates:

$$\text{Current Round Number} = 0 + 1 = 1$$

It immediately initializes a tracking record in `quizapp_quizplaysummery` with status `IN_PROGRESS` and returns 60 questions formatted as follows:

Listing 2.2: Sample Sanitized Question JSON Delivered to Karim's Phone

```
1 [
2   {
3     "id": 1042,
4     "question": "Which river is known as the longest river in
5       Bangladesh?",
6     "option1": "Padma",
7     "option2": "Meghna",
8     "option3": "Jamuna",
9     "option4": "Surma",
10    "category": 1,
11    "weightage": 1
12  },
13  {
14    "id": 2891,
15    "question": "What is the official currency of Japan?",
16    "option1": "Yuan",
17    "option2": "Yen",
18    "option3": "Won",
19    "option4": "Ringgit",
20    "category": 2,
21    "weightage": 2
22  }
23 ]
```

2.3.2 Racing Against the 180-Second Clock

The 180-second visual timer begins ticking down on Karim's screen.

- **Question 1 to 20 (Foundational Tier):** Karim answers 20 general knowledge questions rapidly, needing only 1.8 seconds per question (36 seconds elapsed).
- **Question 21 to 45 (Intermediate Tier):** Karim tackles BCS science and Bangladesh constitutional history. He maintains focus and answers accurately (80 seconds elapsed, 116 seconds total).
- **Question 46 to 60 (Expert Tier):** Karim answers complex literature and world geography trivia under time pressure.

At $t = 165$ seconds, Karim taps the final option. He has answered all **60 questions**, achieving **58 correct answers** with **15 seconds remaining** on his 180-second clock!

Chapter Key Takeaways

- Quizard avoids expensive `ORDER BY RAND()` queries by utilizing two-stage integer ID sampling in memory, reducing question delivery latency to under 50 milliseconds.
- Questions are ordered progressively by difficulty weightage, creating a rewarding difficulty ramp from foundational trivia to expert tie-breakers.
- Each question delivery initializes a verifiable session round on the server, ensuring every answer submission is uniquely paired with an active tracking token.

Chapter 3

The Shield of Integrity: Cryptography, Anti-Cheat, and Score Persistence

The Security Engine in Brief

Because real monetary prizes (e.g., 10,000 BDT to 25,000 BDT) are awarded to top tournament rankers, tournament integrity is paramount. Quizard implements a military-grade security barrier: end-to-end AES-256-CBC encrypted score payloads, server-authoritative identity enforcement, and real-time anti-cheat velocity heuristics that instantly detect bots and clock manipulation.

3.1 The End-to-End Cryptographic Handshake

When a player finishes their 60-question round, the client app does not transmit plain text scores over the network. If plain JSON were sent, any user with basic browser developer tools could intercept the network request and change their score to 60 in 1 second.

Quizard secures the transmission using ****AES-256-CBC Payload Cryptography****:

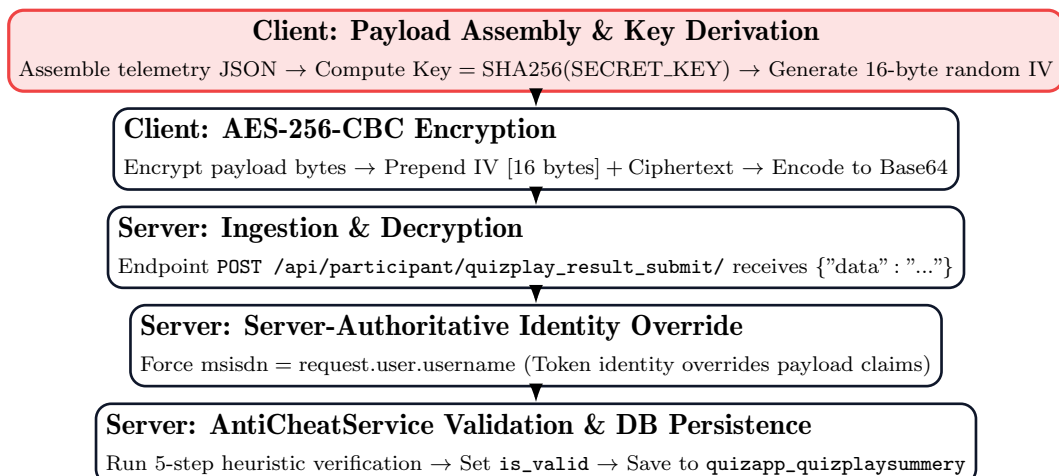


Figure 3.1: End-to-End Cryptographic Scoring & Anti-Cheat Architecture

3.2 The Multi-Layer Anti-Cheat Verification Suite

The server's `AntiCheatService` examines every decrypted play through 5 heuristic checks before accepting a score:

1. Human Velocity & Answering Speed Limit:

$$\text{Answer Velocity} = \frac{\Delta \text{Score}}{\Delta \text{Time Taken}} \leq 1.5 \text{ answers / second}$$

Human cognitive processing requires at least 0.66 seconds to read a question, parse 4 options, and tap the screen. If a submission claims 60 correct answers in 2 seconds, the calculated velocity (30 ans/sec) fails the limit, immediately tagging the record as `SUSPICIOUS` with reason `"Impossible answer speed"`.

2. Timer Consistency & Server Gap Verification:

$$\Delta \text{Time Taken} \leq 10 \times \Delta \text{Server Timestamp Gap}$$

Ensures the client's internal clock did not pause, jump, or accelerate abnormally compared to real-world server time.

3. Timer Rollback Protection:

$$\Delta \text{Time Taken} \geq -10 \text{ seconds}$$

Prevents malicious scripts from resetting or rolling back elapsed time during a live round.

4. Score Monotonic Progression:

$$\Delta \text{Right Count} \geq 0$$

A contestant's cumulative score within a single round cannot decrease between checkpoints.

5. Attempt Flooding & Reset Spam Cap: Caps daily attempts at `MAX_ATTEMPTS_PER_DAY` and detects automated reset loops (> 10 consecutive zero-second resets).

3.3 Two Divergent Journeys: Salma vs. Tanvir

3.3.1 Salma's Legitimate Victory (59/60 in 150 Seconds)

In Sylhet, Salma starts her second tournament attempt of the day. Over 150 seconds, she steadily answers 60 rapid questions across all difficulty tiers.

When the round completes:

1. Her phone generates an encrypted AES-256 payload and submits it to `/api/participant/quizplay_resu`
2. The backend decrypts the payload: `Score: 59, Time Taken: 150s, Round: 2, Event: 34`.
3. `AntiCheatService` computes her answering velocity:

$$\text{Velocity} = \frac{59 \text{ answers}}{150 \text{ seconds}} = 0.393 \text{ answers/sec} \leq 1.5 \implies \text{PASSED}$$

4. The record is saved to `quizapp_quizplaysummery` with `is_valid = 1` and `status = "VALID"`.
5. Salma takes the **#1 Rank** on the live tournament leaderboard!

3.3.2 Tanvir's Automated Exploit Attempt (Flagged & Blocked)

Meanwhile, a tech-savvy user named Tanvir attempts to use an automated browser script to send forged quiz results. His script submits a forged score claiming **60** correct answers in only **2 seconds**.

1. The backend receives and decrypts Tanvir's payload: **Score: 60, Time Taken: 2s**.
2. `AntiCheatService` runs the velocity check:

$$\text{Velocity} = \frac{60 \text{ answers}}{2 \text{ seconds}} = 30.0 \text{ answers/sec} > 1.5 \implies \text{FAILED}$$

3. The backend persists the record with `is_valid = 0`, `status = "SUSPICIOUS"`, and `reason_msg = "Impossible answer speed"`.
4. When the leaderboard queries today's rankings (`WHERE is_valid = 1`), Tanvir's record is completely ignored. Honest players like Salma and Rahim remain protected at the top of the leaderboard!

Player	Score	Time	Velocity	Anti-Cheat Flag	Leaderboard Status
Salma	59 / 60	150s	0.39 ans/sec	VALID (<code>is_valid=1</code>)	Rank 1 (Active)
Rahim	58 / 60	140s	0.41 ans/sec	VALID (<code>is_valid=1</code>)	Rank 2 (Active)
Karim	58 / 60	165s	0.35 ans/sec	VALID (<code>is_valid=1</code>)	Rank 3 (Active)
Tanvir	60 / 60	2s	30.0 ans/sec	SUSPICIOUS (<code>is_valid=0</code>)	Filtered Out (Ignored)

Table 3.1: Comparison of Legitimate vs. Exploitative Score Submissions

Chapter Key Takeaways

- All score submissions are sealed client-side using **AES-256-CBC** cryptography before network transmission.
- The server strictly overrides user identity from the authenticated JWT/Bearer token, preventing MSISDN spoofing.
- `AntiCheatService` applies real-time human speed and timer consistency heuristics, ensuring fraudulent submissions are tagged `is_valid = 0` and excluded from official prize leaderboards.

Chapter 4

The Hall of Champions: Live Leaderboards and Tournament Consolidation

The Tournament Engine in Brief

Quizard's competitive ecosystem operates across two complementary timescales: the ****Real-Time Daily Leaderboard**** (which updates instantly as players submit scores) and the ****Automated Tournament Consolidation Crons**** (which verify subscriber billing, freeze official winners, and distribute cash prizes at 4:30 AM and 5:30 AM every week).

4.1 The Real-Time Daily Leaderboard Engine

Throughout the day, thousands of players compete across Bangladesh. When any contestant opens the leaderboard screen, the backend executes the high-performance query (or stored procedure `sp_quiz_leaderboard_new`):

Listing 4.1: The PostgreSQL Leaderboard Window Aggregation Query

```
1 WITH RankedPlays AS (  
2     SELECT  
3         msisdn,  
4         right_count AS score,  
5         time_taken,  
6         event_id,  
7         portal_id,  
8         user_id,  
9         ROW_NUMBER() OVER (  
10            PARTITION BY msisdn  
11            ORDER BY right_count DESC, time_taken ASC, id ASC  
12        ) AS rn  
13 FROM quizapp_quizplaysummery  
14 WHERE is_valid = TRUE  
15     AND (date = (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Dhaka')::date)  
16     AND (msisdn NOT LIKE '%1798533533%')  
17     AND (:eventId = 0 OR event_id = :eventId)  
18     AND (:portalId = 0 OR portal_id = :portalId)  
19 )  
20 SELECT  
21     msisdn,  
22     score,  
23     time_taken,
```



```

24 DENSE_RANK() OVER (
25     ORDER BY score DESC, time_taken ASC
26 ) AS user_rank
27 FROM RankedPlays
28 WHERE rn = 1
29 ORDER BY user_rank ASC
30 LIMIT 100;

```

4.1.1 Strict Tie-Breaking Hierarchy

When two or more players achieve high scores, Quizard resolves rank positions using a transparent 3-tier hierarchy:

1. **Primary Criterion (Highest Score):** The player with more correct answers ranks higher.
2. **Secondary Criterion (Fastest Time):** If scores are tied (e.g., both Rahim and Karim scored 58/60), the player who finished in fewer seconds ranks higher (Rahim in 140s vs. Karim in 165s $\implies$ Rahim takes Rank 2, Karim takes Rank 3).
3. **Tertiary Criterion (Earliest Submission):** If both score and time are identical, the player whose play completed first receives the higher rank.

4.2 The Overnight Daily Verification Pipeline (04:30 AM Cron)

At 04:30 AM every morning, when gameplay for the previous calendar day has concluded, an automated background cron triggers `ProcessCronDailyWinnerAPIView`:

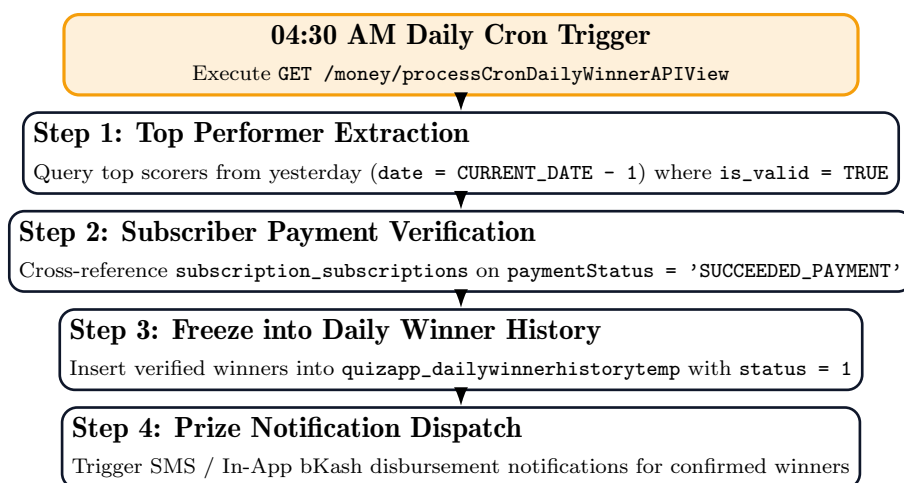


Figure 4.1: The 04:30 AM Daily Winner Verification & Freezing Process

4.3 The Weekly Tournament Championship (Thursday 05:30 AM)

While daily quizzes offer rapid excitement, the premier championship in Quizard is the **Weekly Mega Tournament**.

4.3.1 The 7-Day Rolling Tournament Cycle

The tournament week begins at **00:00:00** on Thursday and concludes at **23:59:59** on Wednesday. Every Thursday morning at **05:30 AM**, the weekly consolidation cron initiates:

GET /money/processCronWeeklyWinnerAPIView

The backend invokes the stored procedure:

```
process_quizard_top_winner_by_date_range_july(event_id, start_date, end_date)
```

4.3.2 Stored Procedure Architecture

The stored procedure consolidates all valid plays across the 7-day window, filters out non-subscribers, assigns official serial numbers, and writes the frozen champions to `quizapp_weeklywinnerhistoryt`

Listing 4.2: Weekly Tournament Winner Processing Stored Procedure

```

1  -- Consolidate 7-day tournament winners and freeze rankings
2  INSERT INTO quizapp_weeklywinnerhistorytemp (
3      msisdn, payer, event_id, service_type, category,
4      right_count, time_taken, serial_no, user_id,
5      status, week_start_date, week_end_date, portal_id, created_at
6  )
7  SELECT
8      q.msisdn,
9      s.payer,
10     q.event_id,
11     'WeeklyTournament' AS service_type,
12     'Rapid60' AS category,
13     q.right_count,
14     q.time_taken,
15     ROW_NUMBER() OVER (ORDER BY q.right_count DESC, q.time_taken ASC)
16                        AS serial_no,
17     q.user_id,
18     1 AS status,
19     v_start_date AS week_start_date,
20     v_end_date AS week_end_date,
21     q.portal_id,
22     NOW() AS created_at
23 FROM quizapp_quizplaysummery q
24 JOIN subscription_subscriptions s ON q.msisdn = s.msisdn
25 WHERE q.event_id = in_event_id
26     AND q.date BETWEEN v_start_date AND v_end_date
27     AND q.is_valid = TRUE
28     AND s.paymentStatus = 'SUCCEEDED_PAYMENT'
29 ORDER BY q.right_count DESC, q.time_taken ASC
30 LIMIT 50;
```

4.4 The Player Journey: Ayesha Wins the Weekly Championship

In Rajshahi, Ayesha has been playing consistently throughout the 7-day tournament cycle on Portal 21 (Robi Quizard).

On Wednesday night at 11:30 PM, Ayesha plays her final allowed round. She achieves a score of ****59 out of 60**** in ****135 seconds****.

1. At 05:30 AM on Thursday, the weekly cron runs.
2. The stored procedure scans the 7-day window ($N = 42,000$ total plays).
3. Ayesha's active subscription status is verified against billing records.
4. She is crowned ****Serial No. 1 Weekly Champion**** with a prize of 25,000 BDT!

5. When Ayesha opens her app on Thursday morning, `GET /money/tWinnerApiFrontend` displays her name and profile at the very top of the Hall of Champions.

Chapter Key Takeaways

- Live leaderboards use SQL window partitioning (`ROW_NUMBER() OVER`) to ensure each player's top round is represented accurately without Frankenstein metric mixing.
- Overnight crons at 04:30 AM and Thursday 05:30 AM verify paid subscriber status, preventing unbilled or fraudulent accounts from claiming official prize money.
- Frozen winner tables ensure immutable audit records for finance disbursement and regulatory compliance.

Chapter 5

Architecture Blueprint: Clean Architecture and Database Schemas

The Architectural Evolution in Brief

As Quizard expands to support millions of concurrent participants across telco partner portals (bKash, Banglalink, Robi), the backend is undergoing an architectural upgrade: transitioning from the original Django monolithic codebase to a high-throughput **.NET 10 Clean Architecture** backed by an enterprise **PostgreSQL** cluster.

5.1 The .NET 10 Clean Architecture Structure

The modernized .NET 10 backend enforces strict layer boundaries and dependency inversion:

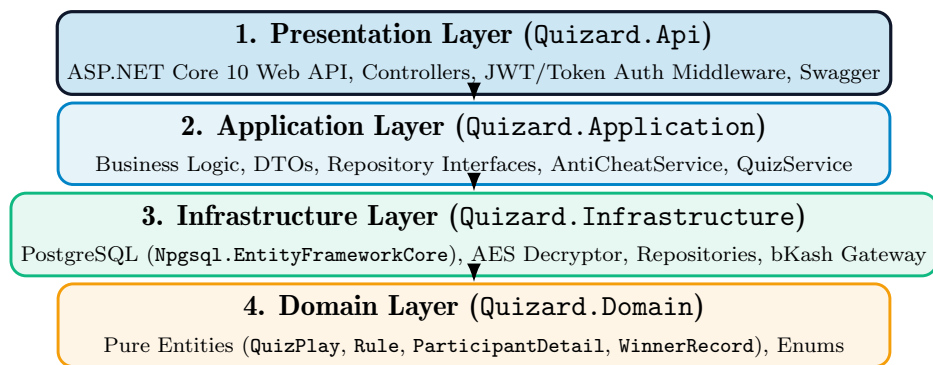


Figure 5.1: Quizard .NET 10 Clean Architecture Dependency Hierarchy

5.2 Comprehensive API Endpoint and Schema Mapping

The following table provides the definitive cross-reference between Django REST endpoints, .NET 10 Clean Architecture controllers, and underlying PostgreSQL tables:

5.3 Critical PostgreSQL Table Schemas and Indexing

To sustain high concurrency during live 60-question tournaments, the PostgreSQL database utilizes optimized multi-column indexes:

Django REST Endpoint	.NET 10 Clean Controller	PostgreSQL Table	Core Functionality
GET /api/rules/	RulesController.Get	cms_rules	Serves allocated time (180s), 60 Qs, and 2-round cap.
GET /api/questions/	QuestionController.Get	cms_questions	Delivers 60 weighted questions with anti-memorization filtering.
POST /api/participant/...	QuizController.Submit	quizapp_quizplaysummery	Ingests AES-256 encrypted scores, runs anti-cheat, saves play.
GET /api/leaderboard/	LeaderboardController.Get	quizapp_quizplaysummery	Computes live daily best attempt rankings per MSISDN.
GET /api/participantLeaderboard	LeaderboardController.GetUser	quizapp_participantdetails	Returns player's personal rank, score, and profile.
GET /money/processCronDaily...	DailyWinnerJob.Execute	quizapp_dailywinner...	04:30 AM cron: verifies billing and freezes daily winners.
GET /money/processCronWeekly...	WeeklyWinnerJob.Execute	quizapp_weeklywinner...	Thursday 05:30 AM cron: consolidates 7-day tournament champions.
GET /money/tWinnerApiFrontend	LeaderboardController.Winner	quizapp_weeklywinner...	Renders official Hall of Champions on frontend.

Table 5.1: Cross-Stack Endpoint & PostgreSQL Database Mapping Matrix

Listing 5.1: Essential PostgreSQL Indexes for Sub-Millisecond Queries

```
1 -- Index for live daily leaderboard lookups
2 CREATE INDEX idx_quizplay_event_date_valid
3 ON quizapp_quizplaysummery (event_id, portal_id, date, is_valid,
4   right_count DESC, time_taken ASC);
5
6 -- Index for player attempt history and anti-memorization lookups
7 CREATE INDEX idx_quizplay_msisdn_date
8 ON quizapp_quizplaysummery (msisdn, date);
9
10 -- Index for weekly tournament consolidation stored procedures
11 CREATE INDEX idx_quizplay_weekly_range
12 ON quizapp_quizplaysummery (event_id, date, is_valid)
13 INCLUDE (msisdn, right_count, time_taken, user_id, portal_id);
```

Chapter Key Takeaways

- .NET 10 Clean Architecture isolates domain business rules from external frameworks and persistence details.
- In-memory mock stores (ScoreStore) have been completely eliminated in favor of direct, verified PostgreSQL database transactions.
- Composite indexing on (event_id, portal_id, date, is_valid) guarantees sub-millisecond query latency even during peak concurrent tournament traffic.

Chapter 6

The Gateway of Speed: The FastAPI Microservice and CMS Engine

The Quizard-API (`cms.quizard.live`) in Brief

While the Django backend and .NET Clean Architecture manage core business transactions, a specialized, asynchronous microservice—**Quizard-API** (hosted in production at `cms.quizard.live`)—powers real-time campaign telemetry, high-throughput MongoDB activity logging, sub-5ms cached rules delivery, and distributed token verification.

6.1 The Need for a Dedicated High-Velocity Microservice

During national tournament campaigns across Bangladesh, hundreds of thousands of concurrent users interact with Quizard through web browsers, the bKash In-App portal, and telecom partner portals (Banglalink, Robi).

Every time a player navigates between screens, clicks a tournament banner, or fetches the latest rules, an HTTP request is generated. If all of this high-frequency traffic were routed directly through traditional relational database queries, relational connection pools would become saturated, introducing latency into critical quiz play and score submission workflows.

To solve this, the engineering team decoupled high-velocity, read-heavy, and event-streaming workloads into **Quizard-API** (built on **FastAPI**, **Motor AsyncIO for MongoDB**, **Redis**, and **PostgreSQL**).

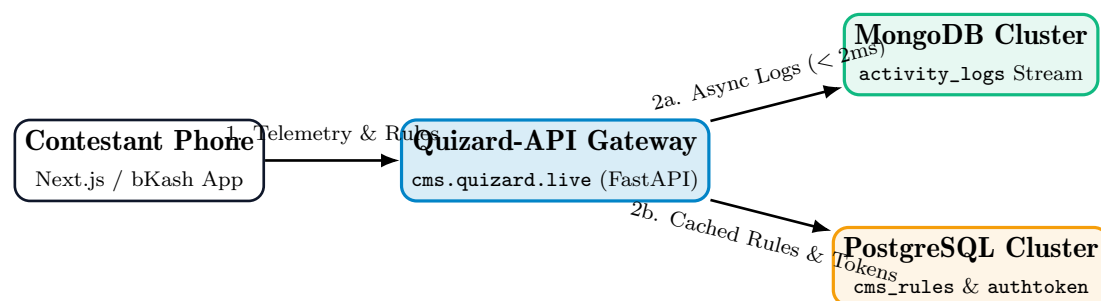


Figure 6.1: Quizard-API (`cms.quizard.live`) Asynchronous Ingestion & Caching Architecture

6.2 Core Responsibilities of Quizard-API

- **High-Velocity Activity Logging (/activity-logs/):** Ingests clickstreams, page views, campaign attribution IDs (cid), IP addresses, user agents, and portal tags into MongoDB asynchronously in under 2 milliseconds.
- **Hourly Aggregation Pipelines (/activity-logs/all_log_hourly):** Executes MongoDB aggregation pipelines to group millions of raw logs into hourly summaries by campaign, portal, and page for executive marketing dashboards.
- **Sub-5ms Cached Rules Serving (/rules/, /rules/cached):** Serves tournament rule configurations (180s timer, 60 questions, 2-round cap) from memory/JSON cache to ensure instantaneous client page loading.
- **Token Authentication Proxy (/tokens/tokens/{key}):** Performs low-overhead token validation for microservices and frontend clients directly against the PostgreSQL `auth_token` table.
- **CMS Media Asset Management (/rules/upload-logo/):** Handles secure multipart uploads for partner logos, banners, and tournament artwork.

6.3 Real-World Stories and Concrete Examples

6.3.1 Story 1: Rahim's Campaign Click and Asynchronous Telemetry

It is 04:00 PM during a major holiday tournament campaign. Rahim is browsing the bKash payment app and sees a promotional banner with campaign tag `cid="bkash_victory_2026"`.

When Rahim taps the banner:

1. His phone opens the Quizard web application and automatically dispatches a non-blocking background telemetry beacon:

POST `https://cms.quizard.live/activity-logs/`

2. The request body contains campaign metadata:

Listing 6.1: Activity Log Ingestion JSON Payload

```
1 {
2   "msisdn": "01711000001",
3   "cid": "bkash_victory_2026",
4   "company": 15,
5   "page_name": "tournament_entry_page",
6   "source_url":
7     "https://quizard.live/tournament?cid=bkash_victory_2026",
8   "useragent": "Mozilla/5.0 (Linux; Android 14; Pixel 8)"
9 }
```

3. **Under 2ms Ingestion:** FastAPI extracts Rahim's client IP (`get_client_ip`), adds the UTC timestamp and hourly partition key (`ymdh`), and writes the record directly to MongoDB:

Listing 6.2: Asynchronous Ingestion Handler in Quizard-API

```
1 @router.post("/")
2 async def create_activity_log(request: Request):
3     ip = get_client_ip(request)
```

```

4     log_data = await request.json()
5     now = datetime.now()
6     log_data["ip"] = ip
7     log_data["created_at"] = now
8     log_data["ymdh"] = now.replace(minute=0, second=0,
        microsecond=0)
9     log_data["date"] = now.strftime('%Y-%m-%d')
10
11     # Instant Non-Blocking MongoDB Async Insertion
12     await activity_logs_collection.insert_one(log_data)
13     return "OK"

```

4. Rahim experiences zero UI lag or stuttering because telemetry ingestion is completely non-blocking and isolated from the transactional database.

6.3.2 Story 2: Marketing Executives Analyze Hourly Campaign ROI

At 05:00 PM, the marketing team at Digital For Good evaluates the performance of the `bkash_victory_2026` campaign.

Instead of executing slow `COUNT(*)` queries on millions of unindexed rows, the analytics dashboard calls:

GET https://cms.quizard.live/activity-logs/all_log_hourly?hour=1

FastAPI executes an optimized MongoDB aggregation pipeline, grouping by campaign ID, page name, and portal:

Listing 6.3: MongoDB Aggregation Pipeline for Hourly Analytics

```

1 pipeline = [
2     { '$match': { 'ymdh': target_hour } },
3     {
4         '$group': {
5             '_id': {
6                 'cid': '$cid',
7                 'company': '$company',
8                 'page_name': '$page_name',
9                 'date': '$date'
10            },
11            'total_impressions': { '$sum': 1 }
12        }
13    }
14 ]
15 result = await
    activity_logs_collection.aggregate(pipeline).to_list(None)

```

The marketing team instantly sees that **48,500 unique contestants** entered the tournament through Rahim's campaign link within that single hour.

6.3.3 Story 3: Karim Fetches Instant Cached Rules

When Karim starts his tournament round in Chittagong, his app requests the active rules from `cms.quizard.live`:

GET https://cms.quizard.live/rules/?portal=15&is_event=true

Quizard-API checks its memory-cached rules snapshot:

- **Duration:** 180 seconds.
- **Question Set:** 60 questions.
- **Round Cap:** 2 attempts per day.
- **Prize:** 10,000 BDT.

The response is delivered to Karim's screen in ****3 milliseconds****, allowing the quiz interface to render immediately without waiting for relational SQL queries.

Endpoint	Method	Data Store	Core Functionality
/activity-logs/	POST	MongoDB	Ingests pageviews, clicks, and campaign attribution in < 2ms.
/activity-logs/all_log	GET	MongoDB	Aggregates daily telemetry across portals and campaigns.
/activity-logs/all_log_hourly	GET	MongoDB	Groups hourly engagement metrics for real-time analytics.
/rules/	GET	PostgreSQL / Cache	Returns cached rules (60 Qs, 180s, 2 rounds) in sub-5ms.
/rules/upload-logo/	POST	Static Disk	Uploads partner logos and campaign banner artwork.
/tokens/tokens/{key}	GET	PostgreSQL	Validates bearer tokens for lightweight microservice clients.

Table 6.1: Quizard-API (`cms.quizard.live`) Microservice Endpoint Directory

Chapter Key Takeaways

- `cms.quizard.live` is powered by ****Quizard-API****, an asynchronous FastAPI microservice built to offload telemetry, impression tracking, and cached rule distribution from the core backend.
- Telemetry logs are stored in ****MongoDB**** using non-blocking async IO, processing thousands of simultaneous events per second without degrading gameplay performance.
- The separation of transactional quiz gameplay (.NET 10 / Django on PostgreSQL) from high-velocity telemetry (FastAPI on MongoDB) ensures scalability, resilience, and sub-50ms responsiveness platform-wide.

Chapter 7

The Master Ecosystem: CMS Operations, Multi-Game Engines, and Telco Billing

The Complete Quizard Ecosystem in Brief

Beyond rapid 60-question trivia, Quizard is an enterprise gaming ecosystem. It features a multi-tenant Content Management System (CMS) with question approval lifecycles, role-based admin governance, a multi-game suite (Wordly, Speed Math, Memory Match, Sudoku), direct telco operator billing (bKash, Banglalink, GP, Robi), affiliate campaign attribution, and automated financial prize disbursement.

7.1 The CMS Question Authoring and Approval Pipeline

High-quality trivia requires rigorous editorial oversight. Quizard's CMS (`apps.cms`) provides a multi-stage question authoring pipeline:

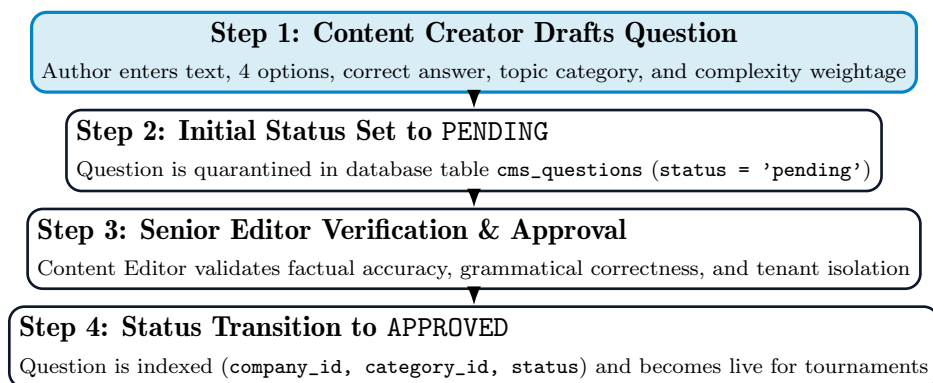


Figure 7.1: The Multi-Stage CMS Question Approval Pipeline

7.1.1 Data Model of a CMS Question Entity

Every question entity is strictly bound to its company tenant and topic category:

Listing 7.1: The Question Entity Data Model

```
1 class Questions(models.Model):
```

```
2     company = models.ForeignKey(Company, on_delete=models.CASCADE)
3     category = models.ForeignKey(Category, on_delete=models.CASCADE)
4     weightage = models.JSONField(default=list) # Difficulty 1 to 6
5     question = models.CharField(max_length=400)
6     options_nub = models.IntegerField(default=4)
7     option1 = models.CharField(max_length=200)
8     option2 = models.CharField(max_length=200)
9     option3 = models.CharField(max_length=200, null=True, blank=True)
10    option4 = models.CharField(max_length=200, null=True, blank=True)
11    ans = models.CharField(max_length=200) # e.g. "Option1"
12    status = models.CharField(choices=APPROVE_STATUS,
                             default='approved')
```

7.2 Admin Governance and Role-Based Access Control (RBAC)

Quizard enforces strict enterprise security using granular Role-Based Access Control (`apps.cms_users`):

- **Super Administrator:** Full access across all portals, database settings, financial payout configurations, and user management.
- **Content Manager / Creator:** Restricted to drafting, categorizing, and editing trivia questions within assigned company portals (e.g., Portal 15 bKash vs. Portal 20 Banglalink).
- **Finance & Operations Auditor:** Read and export access to prize disbursement records (`apps.disbursementReport`), daily charging graphs, and bKash payout reconciliation.
- **Partner Admin (Tenant Isolation):** Restricted views allowing telecom partners (Robi, Banglalink) to inspect only their active subscriber counts and tournament leaderboards.

7.3 The Multi-Game Suite: Beyond Rapid Trivia

Quizard is a multi-genre gaming platform offering diverse casual and competitive titles:

7.3.1 1. Wordly: The Daily Bengali & English Word Puzzle

In `apps.word`, players solve daily 3, 4, 5, and 6-letter word puzzles in both Bengali and English.

- Players have 6 attempts to guess the hidden dictionary word.
- Guess telemetry is tracked daily.
- Stored procedures `WordlyWinner` and `WordlyWeeklyWinner` calculate daily accuracy streaks and award cash prizes to top word puzzle masters.

7.3.2 2. Speed Math and Addition Sprint

In `api/addition`, contestants solve rapid arithmetic challenges against an accelerating timer, testing reflex calculation speed.

7.3.3 3. Memory Match and Sudoku Challenges

In `api/participants`, puzzle enthusiasts compete in card-matching memory tests and tiered Sudoku difficulties (Foundational, Intermediate, Grandmaster).

7.4 Direct Operator Billing (DOB) and Telco Subscriptions

To monetize games across Bangladesh, Quizard integrates directly with Mobile Network Operators (MNOs) and Mobile Financial Services (MFS):



Figure 7.2: Telco Direct Operator Billing (DOB) & Subscription Lifecycle

- **bKash Direct Charging (apps.bkash):** Seamless in-app subscription and one-time tournament passes via tokenized bKash payment APIs.
- **Banglalink Direct Operator Billing (apps.banglalink):** Direct carrier balance deduction with automated renewal crons and unsubscribe postbacks.
- **Grameenphone & Robi Carrier Integrations (apps.grameenphone, apps.webhook):** High-availability webhook listeners capturing charging successes and subscription renewals.

7.5 Financial Operations, Referrals, and Prize Disbursements

Quizard features an automated financial operations suite in `apps.disbursementReport`:

- **Micro-Prize Tiers (5 TK & 10 TK Winners):** Automated stored procedures `5tkWinner` and `10tkWinner` aggregate and verify daily runners-up who achieve qualifying scores.
- **Referral Bonus Rewards (PrizeOnReferral, ReferHistory):** When players invite friends using custom referral codes, bonus points and cash bounties are automatically credited upon the referee's first paid tournament round.
- **Automated bKash Payout Gateway:** Once winners are frozen in `quizapp_dailywinnerhistorytemp` or `quizapp_weeklywinnerhistorytemp`, the finance disbursement engine triggers the bKash Merchant B2C Disbursement API, sending cash directly to the player's bKash wallet.
- **Churn-Back Win-Back Campaigns (ChurnBackCashBack):** Automatically rewards re-activated lapsed subscribers with promotional game passes.

7.6 Ad Network Attribution and Campaign Tracking

In `apps.campaign`, the platform tracks digital marketing acquisition across ad networks:

- **Ad Networks (Adnetwork):** Defines partner ad networks (e.g., Google, Meta, local DSPs), click ID parameters (`click_id`), and server-to-server passback postback URLs.
- **Campaigns (Campaign):** Maps promotional campaigns to target portals, automatically converting USD payout rates to BDT ($\text{Payout}_{\text{BDT}} = \text{Payout}_{\text{USD}} \times 120$) and tracking conversion volumes.

7.7 Real-World Stories: Behind the Scenes

7.7.1 Story 1: Farhana Prepares the National BCS Tournament Questions

In Dhaka, Farhana is a Senior Content Manager for Quizard. Ahead of the upcoming 46th BCS Preliminary Exam, she uses the CMS admin panel to upload 500 new verified questions across Bangladesh Affairs, International Relations, and General Science.

1. She sets category IDs to BCS Prep (`category = 1`) and complexity weightage from 1 to 5.
2. The questions are saved as `status = 'pending'`.
3. Farhana reviews each question for syllabus accuracy, verifies the 4 options, and approves the batch.
4. The questions instantly enter the active question pool for tonight's BCS Mega Tournament!

7.7.2 Story 2: Rupa Wins the Daily Wordly Challenge in Sylhet

In Sylhet, Rupa plays the Bengali 5-letter Wordly challenge on `wordly.quizard.live`. In 4 guesses, she deciphers the hidden 5-letter word **"POTAKA"** (Flag).

1. Her score and elapsed time are recorded.
2. At midnight, the `WordlyWinner` stored procedure runs, verifying her paid bKash subscription and high accuracy.
3. Rupa wins 500 BDT in prize cash!

7.7.3 Story 3: Monir Audits and Disburses Weekly Prizes

On Thursday at 10:00 AM, Monir, the Operations Finance Lead at Digital For Good, logs into the Disbursement Report portal.

1. He opens `/report/weeklyWinner.html` to inspect the frozen winners generated by the 05:30 AM weekly cron.
2. He cross-references subscriber payment counts and clicks "Approve Payout".
3. The bKash B2C disbursement gateway executes, and within seconds, Ayesha in Rajshahi receives an SMS: *"You have received 25,000 BDT from Quizard Tournament!"*

Chapter Key Takeaways

- Quizard's CMS guarantees question quality and factual accuracy through a rigorous multi-stage approval workflow.
- The platform powers a diverse gaming portfolio including trivia, Wordly word puzzles, speed math, and Sudoku.
- Direct carrier billing across bKash, Banglalink, Grameenphone, and Robi ensures seamless monetization across all major telecom networks.
- Financial disbursement reports and automated bKash B2C payouts provide transparent, audited, and instant cash prize delivery to verified winners.

Chapter 8

The Financial and Analytics Hub: Reporting Engine, Payouts, and Business Intelligence

The Reporting Engine in Brief

Transparency and financial precision are essential for operating a national real-money tournament platform. Quizard features an enterprise Reporting and Business Intelligence (BI) engine (`apps.disbursementReport`, `apps.allReportGraph`, `reporting_app`) that consolidates daily player rankings, validates telco subscriber payments, triggers automated bKash B2C cash disbursements, and renders real-time revenue and retention analytics.

8.1 The Architecture of the Reporting Engine

The Reporting Engine unifies data from live gameplay (`quizapp_quizplaysummery`), carrier billing logs (`subscription_subscriptions`), and payment webhooks into actionable operational reports and automated financial payout workflows.



Figure 8.1: The Automated Prize Disbursement and Financial Reconciliation Lifecycle

8.2 Core Modules of the Reporting Suite

8.2.1 1. Daily and Weekly Tournament Winner Reports

- **Daily Winner Report** (`DailyWinnerViewSet`, `/report/dailyWinner.html`): Lists verified top scorers from yesterday, cross-referenced with successful carrier billing status (`paymentStatus = 'SUCCEEDED_PAYMENT'`).
- **Weekly Mega Championship Report** (`WeeklyWinnerViewSet`, `/report/weeklyWinner.html`): Audits the 7-day tournament leaderboard across all active rules (`Event 34`, `Event 150`) with customizable minimum payment count filters (`payment_count ≥ 3`).

8.2.2 2. Micro-Prize Tier Reporting (5 TK and 10 TK Bounties)

To reward broad participation beyond the top 3 grand champions, Quizard executes micro-prize reward tiers:

- **5 TK Winner Suite** (`5tkWinner`, `FiveTkReport`): Stored procedures identify runners-up who achieve threshold scores (e.g., $\text{Score} \geq 25$) within specific rank brackets (e.g., Ranks 4 to 100).
- **10 TK Winner Suite** (`10tkWinner`, `TenTkWinnerSummaryViewSet`): Consolidates high-accuracy players eligible for 10 BDT wallet top-ups.

8.2.3 3. Wordly Word Puzzle Winner Analytics

In `apps.disbursementReport`, dedicated endpoints (`WordlyDailyWinnerViewSet`, `WordlyDailyDisbursementViews`, `WWeeklyWinner`) audit Bengali and English daily word puzzle accuracy streaks, ensuring Wordly champions are verified before prize payout.

8.2.4 4. Referral Tracking and Bounties (`PrizeOnReferral`)

When existing contestants invite peers using referral codes:

- The referral engine tracks successful onboarding through `report_referhistory`.
- `PrizeOnReferral` generates automated referral bonus reports, crediting cash bounties once the referee completes their first paid tournament attempt.

8.2.5 5. Churn-Back and Win-Back Analytics (`ChurnBackReportViewSet`)

Analyzes lapsed subscribers re-engaging with the platform, calculating promotional return-on-investment (ROI) through stored procedure `ChurnBackCashBack`.

8.2.6 6. Real-Time Revenue and Charging Graphs (`apps.allReportGraph`)

Powers administrative visual analytics through `graphDataApi`:

- **Daily Revenue Trends** (`DailyCharging`): Real-time revenue graphs tracking carrier billing across bKash, Banglalink, Grameenphone, and Robi.
- **Paid Subscriber Growth** (`NewlyJoinedPaidUser`): Funnel analysis of new registrations converting to paying subscribers.
- **DAU and Play Volume**: Hourly volume of tournament rounds played nationwide.

8.3 Real-World Scenario: Tanima Audits and Executes Weekly Payouts

In Dhaka, Tanima is the Head of Financial Operations at Digital For Good. Every Thursday morning at 09:30 AM, she oversees the national tournament prize disbursement:

1. **Leaderboard Verification**: Tanima opens `/report/weeklyWinner.html`. She reviews the frozen winner roster generated by the 05:30 AM weekly cron:
 - **Rank 1**: Ayesha in Rajshahi (59/60 in 135s, 7 successful billing cycles) $\implies$ **25,000 BDT**.

- **Rank 2:** Salma in Sylhet (59/60 in 150s, 7 successful billing cycles) $\Rightarrow$ **10,000 BDT.**
 - **Rank 3:** Rahim in Dhaka (58/60 in 140s, 7 successful billing cycles) $\Rightarrow$ **5,000 BDT.**
 - **Ranks 4–50:** Micro-prize recipients (5 TK and 10 TK bonuses).
2. **Automated bKash Batch Disbursement:** With one click, Tanima authorizes the payout batch. The backend connects directly to the bKash Merchant B2C Disbursement API:

Listing 8.1: bKash B2C Merchant Payout Request Structure

```

1 payload = {
2     "merchantInvoiceNumber":
3         f"QUIZARD-WEEKLY-{event_id}-{winner.id}",
4     "receiverMSISDN": winner.msisdn,
5     "amount": winner.prize_amount,
6     "currency": "BDT",
7     "intent": "tournament_prize_disbursement"
8 }
9 response = requests.post(f"{BKASH_API_URL}/b2cPayment",
10                          json=payload, headers=auth_headers)

```

3. **Instant Confirmation:** Within seconds, bKash debits the merchant disbursement account and credits Ayesha's, Salma's, and Rahim's mobile wallets. Automated SMS notifications confirm the payouts, and immutable transaction IDs are recorded in `UserMigrationAudit`.

Reporting Endpoint	Method	Data Model / View	Operational Function
/report/dailyWinner.html	GET	DailyWinnerViewSet	Audits yesterday's verified top daily winners.
/report/weeklyWinner.html	GET	WeeklyWinnerViewSet	Consolidates 7-day tournament grand prize winners.
/report/fivetkWinner.html	GET	FiveTkWinnerViewSet	Calculates 5 TK micro-prize participant bounties.
/report/wordlyDailytk.html	GET	WordlyDailyWinner...	Reviews Wordly daily puzzle accuracy streaks.
/report/referral.html	GET	PrizeOnReferral	Audits player referral bounties and bonuses.
/api/graphDataApi/...	GET	DailyCharging	Renders visual charts for daily revenue and DAU.

Table 8.1: Comprehensive Reporting & Financial Analytics Directory

Chapter Key Takeaways

- The Reporting Engine bridges competitive gameplay with carrier billing, ensuring prize disbursements are awarded exclusively to legitimate, billed subscribers.
- Multi-tier prize reports support grand tournament awards as well as 5 TK and 10 TK micro-prize participation incentives.
- Automated integration with the bKash Merchant B2C API delivers audited cash prizes directly to contestants' digital wallets in real time.
- Real-time graphical analytics provide management with instant visibility into daily active users, subscription growth, and revenue performance across all telecom portals.

Chapter 9

The Performance Rating Architecture: 6 Score Bands, Dynamic Deltas, and Pro- gression Modeling

The Rating Engine in Brief

A contestant's skill rating is bounded strictly as an **integer on the 0 to 60 tournament scale** ($0 \leq R \leq 60$). Instead of raw score spikes or decimal fractions, Quizard implements an integer-based asymmetric dynamic rating engine. Every round score ($S \in \{0, 1, \dots, 60\}$) dynamically pulls the player's integer rating up or down, directly determining their active competitive rank tier.

9.1 The 6 Integer Performance Bands (0 to 60 Scale)

A player's integer skill rating $R \in \{0, 1, \dots, 60\}$ maps directly to six distinct rank tiers:

Tier	Rank Band Title	Integer Rating Range (R)	Color Theme	Competitive Meaning
1	Novice	0 – 19	Cool Slate Gray	Casual participant / frequent drop-outs
2	Apprentice	20 – 29	Emerald Green	Developing player / building foundation
3	Specialist	30 – 39	Cyan Blue	Consistent mid-tier competitor
4	Expert	40 – 49	Royal Blue	High-level tournament contender
5	Master	50 – 59	Deep Violet	Tournament elite ($\geq 83\%$ accuracy)
6	Grandmaster	60	Imperial Crimson	Flawless perfection (100% accuracy)

Table 9.1: The Quizard 6-Tier Integer Performance Bands (0 to 60 Scale)

9.2 The Integer Dynamic Rating Update Formula

Upon completing a round with raw score $S_{t+1} \in \{0, \dots, 60\}$, the integer delta ΔR is calculated from the score gap $\Delta S = S_{t+1} - R_t$:

$$\Delta R = \begin{cases} 0 & \text{if } S_{t+1} = R_t \\ \max(1, \text{round}(0.20 \cdot (S_{t+1} - R_t))) & \text{if } S_{t+1} > R_t \\ \min(-1, \text{round}(0.35 \cdot (S_{t+1} - R_t))) & \text{if } S_{t+1} < R_t \text{ and } S_{t+1} \geq 20 \\ \min(-2, \text{round}(0.45 \cdot (S_{t+1} - R_t))) & \text{if } S_{t+1} < 20 \text{ (Severe Drop Penalty)} \end{cases} \quad (9.1)$$

The updated integer rating R_{t+1} is clamped strictly between 0 and 60:

$$R_{t+1} = \text{clamp}(0, 60, R_t + \Delta R) \quad (9.2)$$

9.2.1 Core Principles of the Integer Engine

- **Strict Integer Scale** ($0 \leq R \leq 60$): All ratings, deltas, and score bands are whole integers, ensuring clean display on mobile badges and profile cards.
- **Guaranteed Movement**: When a player outperforms their rating ($S > R$), they gain at least +1 rating point. When they underperform ($S < R$), they lose at least -1 point.
- **Asymmetric Penalty**: Quitting or scoring < 20 triggers an aggressive integer penalty (≥ -2 up to -10), preventing casual resets.

9.3 Step-by-Step Scenarios: The Player at Rating 34

To illustrate the integer mechanics, consider a player, Mahir, with an established integer rating of $R_0 = 34$ (**Specialist** Band: 30–39).

Case	Start R	Score (S)	Gap (ΔS)	Formula Math	New R	Resulting Band & Tier Movement
A	34	32	-2	$\text{round}(0.35 \times -2) = -1$	33	Specialist (30–39): Soft -1 integer step.
B	34	36	+2	$\max(1, \text{round}(0.20 \times 2)) = +1$	35	Specialist (30–39): Steady +1 integer gain.
C	34	38	+4	$\text{round}(0.25 \times 4) = +1$	35	Specialist (30–39): Approaching Expert line.
D	34	52	+18	$\text{round}(0.25 \times 18) = +5$	39	Specialist : Major +5 surge to the cusp of Expert.
E	34	12	-22	$\text{round}(0.45 \times -22) = -10$	24	Apprentice (20–29): Demoted after -10 drop!

Table 9.2: Single-Round Integer Transitions for a Player Starting at Rating 34

9.3.1 Detailed Analysis of Each Case

1. Case A: Score 32 (Slightly Below Rating):

$$\Delta S = 32 - 34 = -2 \implies \Delta R = -1 \implies R_1 = 34 - 1 = \mathbf{33}$$

Outcome: Rating gently softens from 34 to 33, remaining securely within the Specialist band (30 – 39).

2. Case B: Score 36 (Slightly Above Rating):

$$\Delta S = 36 - 34 = +2 \implies \Delta R = +1 \implies R_1 = 34 + 1 = \mathbf{35}$$

Outcome: Beating current expectation awards +1 point, climbing to 35.

3. Case C: Score 38 (Strong Specialist Round):

$$\Delta S = 38 - 34 = +4 \implies \Delta R = +1 \implies R_1 = 34 + 1 = \mathbf{35}$$

Outcome: High accuracy sustains upward momentum toward the 40 Expert boundary.

4. Case D: Score 52 (Master Breakout Performance):

$\Delta S = 52 - 34 = +18 \implies \Delta R = \text{round}(0.25 \times 18) = +5 \implies R_1 = 34 + 5 = \mathbf{39}$

Outcome: Scoring 52 correct produces an immediate +5 leap to 39. In the next round, if Mahir scores 55:

$\Delta S = 55 - 39 = +16 \implies \Delta R = +4 \implies R_2 = 39 + 4 = \mathbf{43}$ (Promoted to Expert!)

5. Case E: Score 12 (Severe Drop / Abandonment):

$\Delta S = 12 - 34 = -22 \implies \Delta R = \text{round}(0.45 \times -22) = -10 \implies R_1 = 34 - 10 = \mathbf{24}$

Outcome: Quitting early triggers the severe -10 integer penalty, immediately demoting the player from 34 down to 24 (Apprentice).

9.4 Multi-Round Lifecycle Simulation (Pure Integers 0 to 60)

The table below traces Mahir’s integer rating evolution across 9 consecutive tournament matches:

Round	Start R	Score (S)	Gap (ΔS)	Delta (ΔR)	End R	Tier Status & Progression Description
1	20	25	+5	+1	21	Apprentice (Initial onboarding match)
2	21	36	+15	+3	24	Apprentice (Climbing through Tier 2)
3	24	48	+24	+5	29	Apprentice (Knocking on Specialist door)
4	29	45	+16	+3	32	Specialist (≥ 30 promotion to Tier 3!)
5	32	54	+22	+5	37	Specialist (High-accuracy performance)
6	37	56	+19	+5	42	Expert (≥ 40 promotion to Tier 4!)
7	42	58	+16	+4	46	Expert (Solidifying position in Tier 4)
8	46	60	+14	+4	50	Master (≥ 50 elite status unlocked!)
9	50	60	+10	+3	53	Master (Consolidating Master ranking)

Table 9.3: Multi-Round Simulation Showing Continuous Integer Rating Evolution (0 to 60)

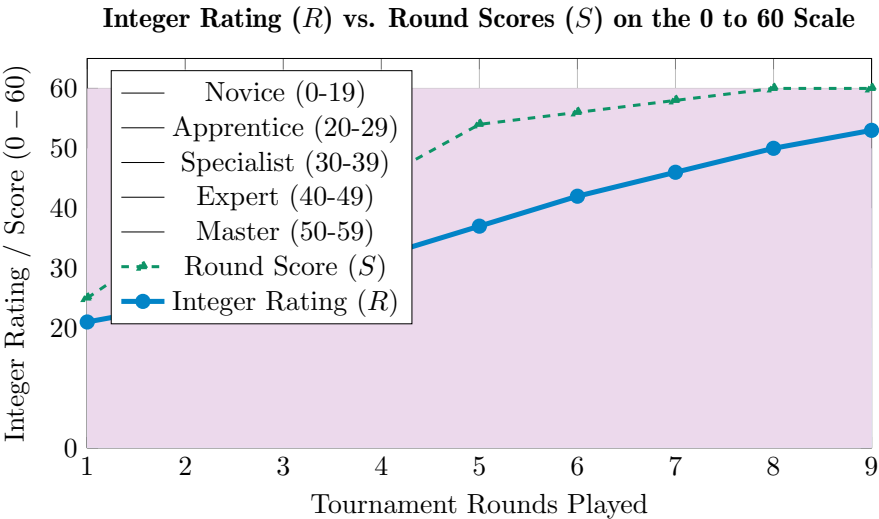


Figure 9.1: Visual Trajectory of Integer Rating ($R \in \{0, \dots, 60\}$) Tracking Round Scores (S)

9.5 Database Schema and API Architecture

9.5.1 1. PostgreSQL Historical Rating Table

Listing 9.1: PostgreSQL Schema for Integer Rating History

```

1 CREATE TABLE participant_rating_history (
2     id BIGSERIAL PRIMARY KEY,
3     msisdn VARCHAR(15) NOT NULL,
4     portal_id INT NOT NULL DEFAULT 15,
5     event_id INT NOT NULL,
6     round_score INT NOT NULL, -- 0 to 60
7     rating_before INT NOT NULL CHECK (rating_before BETWEEN 0 AND 60),
8     -- e.g. 34
9     rating_delta INT NOT NULL, -- e.g. +1, -1, +5, -10
10    rating_after INT NOT NULL CHECK (rating_after BETWEEN 0 AND 60),
11    -- e.g. 35 (strictly integer 0 to 60)
12    band VARCHAR(30) NOT NULL, -- 'Novice', 'Apprentice',
13    'Specialist', 'Expert', 'Master', 'Grandmaster'
14    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
15 );
16 CREATE INDEX idx_rating_history_lookup ON participant_rating_history
17 (msisdn, portal_id, created_at DESC);

```

9.5.2 2. REST Endpoints Specification

Endpoint	Method	Payload / Response Description
/api/participantProfile/ rating_history/	GET	Returns historical time-series of integer ratings (0 – 60), round scores, deltas, and current band title.

Table 9.4: Rating History REST Endpoint

Chapter Key Takeaways

- The rating R and band boundaries are strictly integers between 0 and 60, providing clean, intuitive alignment with the 60-question game format.
- Dynamic integer rounding guarantees immediate feedback ($\geq \pm 1$ point on performance delta) while preventing fractions or decimals.
- Consistent high-accuracy tournament scores elevate contestants to Master (50 – 59) and Grandmaster (60).

Chapter 10

The XP Ledger, Linear Daily Streaks, and Milestone Achievement System

The Retention Gamification Engine in Brief

A 15-day live data review revealed that the majority of players churn after a single session. To cultivate enduring daily return habits, Quizard implements an Experience Point (XP) system powered by an immutable transaction ledger, linear daily streak rewards, a giant 15th-day milestone surge, and collectible achievement badges.

10.1 Core Experience Point (XP) Earning Mechanics

XP serves as Quizard's primary engagement currency, separate from competitive tournament scores. It is awarded for participation and daily consistency:

- **Daily First-Play Bonus:** +50 XP awarded on the contestant's first tournament round of the day.
- **Round Completion Base XP:** +10 XP awarded upon completing each 60-question match.
- **Answer Accuracy Bounty:** +1 XP per correct answer ($\text{RightCount} \leq 60$).

10.2 Linear Daily Streaks with the Giant 15th-Day Surge

To foster an intuitive and predictable daily habit loop, the daily streak progression is strictly **linear**, rewarding each consecutive return day with an incremental boost, culminating in a **giant milestone bump on the 15th day**.

10.2.1 Linear Daily Formula

For Days 1 through 14:

$$\text{Streak Bonus XP}(\text{Day } n) = 20 \times n \quad (10.1)$$

10.2.2 The 15-Day Milestone Progression Matrix

10.3 Milestone Achievement Badges Shelf

Contestants unlock permanent collectible badges displayed on their public profiles:

Consecutive Day	Linear Base Bonus	Special Milestone Surge	Total Daily Streak Reward
Day 1	+20 XP	–	+20 XP
Day 2	+40 XP	–	+40 XP
Day 3	+60 XP	–	+60 XP
Day 4	+80 XP	–	+80 XP
Day 5	+100 XP	–	+100 XP
Day 6	+120 XP	–	+120 XP
Day 7	+140 XP	Week 1 Streak Badge	+140 XP + Bronze Badge
Day 8	+160 XP	–	+160 XP
Day 9	+180 XP	–	+180 XP
Day 10	+200 XP	–	+200 XP
Day 11	+220 XP	–	+220 XP
Day 12	+240 XP	–	+240 XP
Day 13	+260 XP	–	+260 XP
Day 14	+280 XP	–	+280 XP
Day 15	+300 XP	+2,500 XP Milestone Surge	+2,800 XP + Half-Month Titan
Day 16	+320 XP	–	+320 XP

Table 10.1: Linear Daily Streak Progression with Giant 15th-Day Milestone Bump

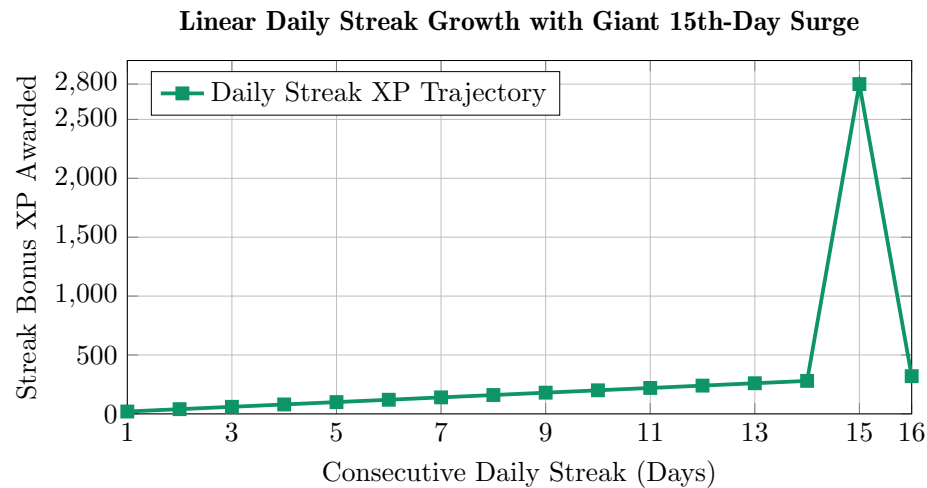


Figure 10.1: Visual Trajectory of Daily Streak Rewards Illustrating the Day 15 Giant Bump

Badge Name	Tier	Unlock Criteria
First Blood	Bronze	Win 1st tournament match or place in the daily top 100.
Week 1 Champion	Bronze	Maintain an unbroken 7-day daily gameplay streak.
Speed Demon	Silver	Complete a 60-question round with ≥ 50 correct in < 90 seconds.
Quiz Scholar	Gold	Play all 4 official games (Daily, Sports, BCS, Tournament) in 7 days.
Century Club	Gold	Accumulate 100 total completed tournament rounds.
Half-Month Titan	Platinum	Complete the 15-day unbroken daily streak and claim the giant surge.
Master Strategist	Diamond	Achieve and maintain a rolling score band in Master (50 – 59).
Grandmaster Elite	Diamond	Score a perfect 60/60 in a live tournament championship.

Table 10.2: Quizard Milestone Achievement Badges Matrix

10.4 Database Schema and API Architecture

10.4.1 1. PostgreSQL Immutable XP Ledger Table

Listing 10.1: PostgreSQL Schema for XP Ledger

```
1 CREATE TABLE participant_xp_ledger (  
2     id BIGSERIAL PRIMARY KEY,  
3     msisdn VARCHAR(15) NOT NULL,  
4     portal_id INT NOT NULL DEFAULT 15,  
5     xp_amount INT NOT NULL,  
6     source_event VARCHAR(50) NOT NULL, -- 'daily_login',  
        'streak_linear', 'streak_day_15_giant_bump', 'quiz_round'  
7     balance_after INT NOT NULL,  
8     created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
9 );  
10 CREATE INDEX idx_xp_ledger_lookup ON participant_xp_ledger (msisdn,  
    portal_id, created_at DESC);
```

10.4.2 2. REST Endpoints Specification

Endpoint	Method	Payload / Response Description
/api/participantProfile/ xp_summary/	GET	Returns cumulative XP, current unbroken streak count, and unlocked badge objects.

Table 10.3: XP Summary REST Endpoint

10.5 Real-World Scenario: Sabina’s 15-Day Streak Journey

In Dhaka, Sabina commits to playing Quizard every morning during her MRT transit:

- Days 1–5:** Sabina earns +20 XP, +40 XP, +60 XP, +80 XP, and +100 XP alongside her daily first-play bonuses.
- Day 7:** Unlocks the **Week 1 Champion** bronze badge.
- Days 8–14:** Her streak climbs linearly (+160 XP → +280 XP).
- Day 15 (The Giant Surge):** Sabina logs in and completes her morning round. The backend triggers the giant 15th-day milestone, crediting **+2,800 XP**, elevating her to Player Level 18, and awarding the coveted **Half-Month Titan** platinum badge.

Chapter Key Takeaways

- The linear streak model (+20 XP × day) provides clear daily progression.
- The giant 15th-day milestone surge (+2,800 XP) delivers a major gratification climax that dramatically curbs mid-month churn.
- The immutable XP ledger guarantees auditable, tamper-proof tracking of all gamification rewards.

Chapter 11

Rank Velocity, Profile Personalization, and Adaptive Live Experiences

Advanced Engagement Architecture in Brief

Beyond static leaderboard snapshots and generic home screens, modern competitive gaming requires dynamic feedback loops. Quizard introduces real-time leaderboard rank-velocity indicators, server-driven personalized profile messaging, and adaptive difficulty modes designed to maximize player retention across its multi-game ecosystem.

11.1 Real-Time Leaderboard Rank Velocity Indicators

Leaderboards can feel static if contestants cannot observe the immediate impact of their latest tournament attempts. Quizard introduces visual velocity indicators:

- **Surging Upward (↑ 5 in Emerald Green):** Indicates the player climbed 5 positions on today's leaderboard since their prior attempt.
- **Slipping Downward (↓ 2 in Crimson Red):** Indicates the player was overtaken by competing participants.
- **Holding Position (= in Slate Gray):** Indicates steady equilibrium.
- **Points to Next Tier Gap:** Computes and displays the exact score difference needed to break into the Top 10:

$$S_{\text{gap}} = \max(1, S_{\text{rank } 10} - S_{\text{player}} + 1) \quad (11.1)$$

11.2 Server-Driven Profile Personalization

Data audits revealed that unsubscribed players achieve high accuracy but churn because they are unaware of cash prize eligibility, and 96.6% of active players only ever play one game. The server evaluates player metrics on every profile request and serves targeted action cards:

11.3 Adaptive Difficulty and High-Experience Fast Timers

For contestants who achieve sustained Master and Grandmaster ratings across consecutive tournaments, Quizard introduces an optional Adaptive Challenge Mode:

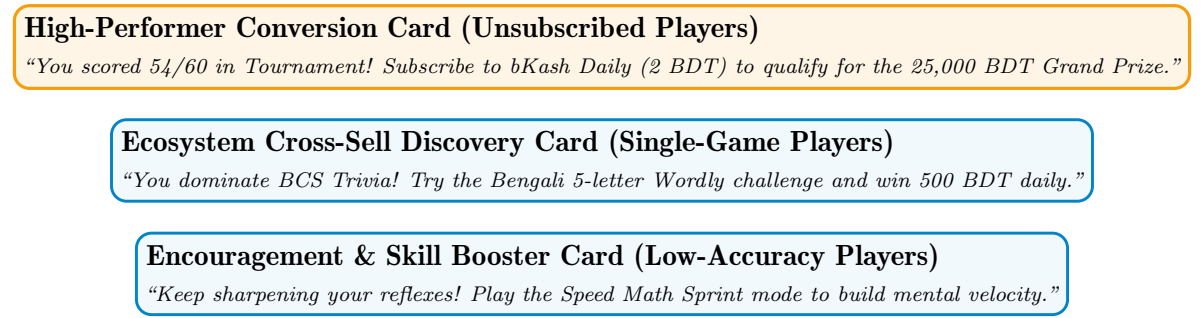


Figure 11.1: Dynamic Server-Driven Profile Personalization Cards

- **Per-Question 10-Second Countdown:** Replaces the total 180s round timer with an aggressive 10-second per-question timer.
- **Automatic Progression:** If unanswered within 10 seconds, the question auto-submits as incorrect and immediately advances.
- **Grandmaster Rating Bonus:** Successful completion under adaptive mode awards an additional +5 rating premium.

11.4 Database Schema and API Architecture

11.4.1 1. REST Endpoints Specification

Endpoint	Method	Payload / Response Description
/api/participantProfile/user_leaderboard/	GET	Returns player rank with rank_delta (↑ / ↓ / =) and points_to_top10.
/api/participantProfile/playing_history/	GET	Returns match history alongside server-generated personalized_message.

Table 11.1: Leaderboard Velocity and Personalization REST Endpoints

11.5 Real-World Scenario: Rafiq’s Leaderboard Surge

In Khulna, Rafiq plays his second daily attempt at 08:30 PM:

1. Rafiq scores 56/60 in 115 seconds.
2. When he opens the leaderboard, a glowing ↑ 4 appears beside his name as he leaps from Rank 12 to Rank 8.
3. The banner beneath his rank displays: “You are only 2 points away from the Top 5!”
4. Below his stats, a personalized card recommends: “You have 1 attempt remaining to-day—play now to secure your weekly champion prize!”

Chapter Key Takeaways

- Rank-velocity indicators (↑ / ↓) create visceral competitive feedback after every completed match.
- Server-driven personalization cards convert high-performing unsubscribed contestants and drive cross-sell traffic across all 4 quiz genres.
- Adaptive fast-timer modes provide high-skill players with engaging grandmaster challenges.

Chapter 12

Technical Glossary and Reference Guide

12.1 Glossary of Core Domain Entities

MSISDN: Mobile Station International Subscriber Directory Number (the player's unique mobile phone number in Bangladesh, e.g., 01711000001).

Event / Rule ID: Unique numeric identifier for a tournament competition rule set (e.g., Event 34 = Daily Rapid Tournament, Event 75 = Mega Weekly Championship).

Portal ID: Tenant identifier providing multi-tenant isolation (e.g., 15 = Quizard Web / bKash, 20 = Banglalink, 21 = Robi).

AES-256-CBC: Advanced Encryption Standard with 256-bit key length operating in Cipher Block Chaining mode, securing all client score transmissions.

Dense Rank: A database ranking function where identical scores receive identical rank numbers without creating gaps in consecutive rank assignments.

Velocity Check: Anti-cheat validation ensuring the rate of answered questions does not exceed human biological reaction thresholds (≤ 1.5 answers/sec).

Final Architecture Conclusion

Quizard's architecture represents a gold standard for high-performance, real-time gaming in emerging markets. By coupling dynamic rules governance, sub-50ms question sampling, end-to-end cryptographic integrity, and automated financial verification, the platform delivers an engaging and trustworthy experience to millions of contestants.